CHAOS Hackathon — PCOS Symptom Guide: Team Plan & Checklists

Oct 15, 2026 · @CA- Studio

1. Project overview

We are building a web app that lets a woman describe her symptoms in her own words and language, then explains them in plain language and tells her what kind of care to seek. Working name: **Sakhi** (rename freely). Challenge: CHAOS #3, Women's Health — Understanding Symptoms, with a focus on PCOS.

The problem. Many women in India live with irregular periods, acne, weight gain or excess hair growth for years without knowing these can be linked to PCOS. Information online is in English, full of jargon, or frightening. Many describe symptoms in Hindi, a regional language or Hinglish, and never reach the right doctor.

Who it is for. Young women (16–35) and their families, on a phone, with patchy English and little time.

What the app does (the MVP):

1. Accepts symptoms as typed text or a voice note, in English, Hindi, a regional language, or a mix ("3 mahine se periods nahi aaye, face pe acne").
2. Pulls out the details: which symptoms, how long, how severe, cycle pattern.
3. Runs a red-flag safety check first. Danger signs skip the explanation and go straight to "seek urgent care now".
4. Explains in plain language, in her language, what these symptoms are *commonly associated with*, grounded in our curated sources and citing them.
5. Tells her which kind of care fits (gynaecologist, general doctor, endocrinologist) and how soon.
6. Produces a one-page doctor-visit summary she can print or show, with questions to ask.
7. Stretch: a symptom and cycle diary saved in MongoDB that shows patterns over weeks.

What it must never do:

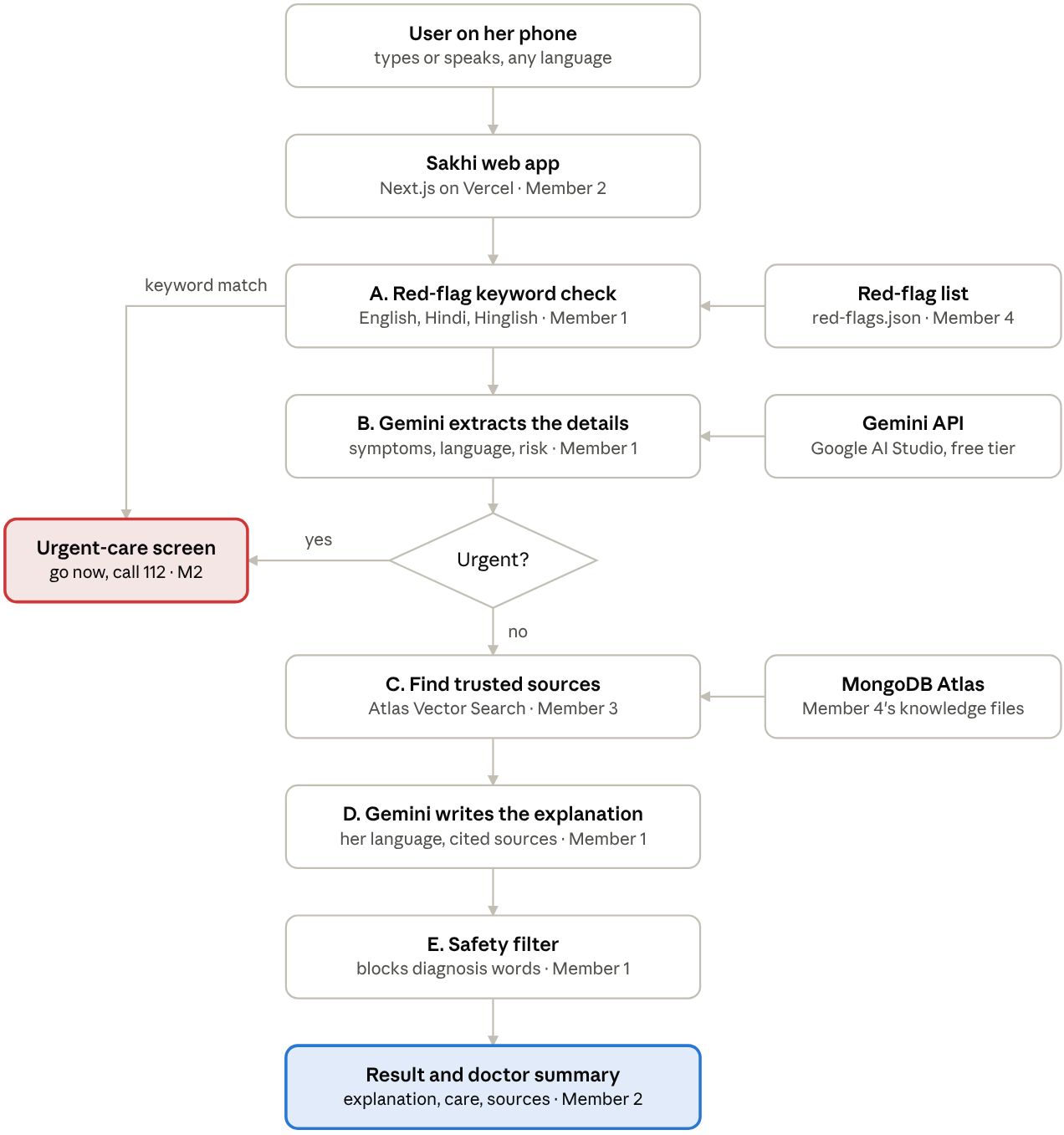
- Never say "you have PCOS", "diagnosis" or "confirmed". It explains possibilities and points to a doctor.
- Never recommend medicines, doses or stopping treatment.
- Never give an answer that isn't grounded in our curated sources.
- Never store personal health data without consent.

What success looks like at the demo: a judge speaks a Hinglish symptom description into a phone and within about 10 seconds gets a clear, kind, cited explanation in Hinglish, a care recommendation and a printable summary. A red-flag input is escalated every single time.

2. How the product works

One request travels through five steps, and the safety check comes first. If anything looks dangerous, the user sees the urgent-care screen and no explanation is generated.

Every answer passes a red-flag check before any explanation



Sakhi request flow · 5 steps, 1 safety decision

A keyword match at step A or a risk judgement at step B both lead to the urgent screen. Otherwise the app finds trusted sources, writes the answer only from them, and filters it before showing it.

What the user experiences:

1. Opens the site on her phone; sees what Sakhi is and that it is not a diagnosis.
2. Types or speaks her symptoms in any supported language.
3. Sees "We heard: …" and, if needed, answers 1–2 follow-up questions as tappable chips.
4. Gets a result: urgency badge, plain explanation, what it is commonly associated with, who to see and when, sources.
5. Taps "Doctor summary" to print or save a one-page summary to show her doctor.
6. Optionally saves the session anonymously, or deletes everything with one tap.

3. Tech stack and free tools

Everything below is free. One Next.js app on Vercel holds both the frontend and the server routes, so there is only one thing to deploy.

Layer	Tool	Why	Owner
Frontend	Next.js (App Router) + TypeScript + Tailwind CSS	One project for UI and API, fast to build, mobile-first	Member 2
Server routes	Next.js route handlers (`/api/analyze` , `/api/summary` , `/api/diary`)	No separate backend; API key stays on the server	Member 1
AI	Gemini API via Google AI Studio (free key), `@google/genai` SDK	Understands Indian languages and voice clips directly; returns structured JSON	Member 1
Embeddings	Gemini embedding model	Turns knowledge-base text into vectors for search	Member 3
Database	MongoDB Atlas M0 (free, 512 MB)	Knowledge chunks, vector search, sessions, diary	Member 3
Search	Atlas Vector Search	Finds the right source passages for each answer	Member 3
Hosting	Vercel Hobby (free)	Push to `main` = live in about 1 minute	Member 1
Domain	Our .xyz domain, pointed at Vercel	Required for submission	Member 1

Layer	Tool	Why	Owner
Code	GitHub repo (team lead creates, adds collaborators)	Required for submission	Everyone
AI coding help	Claude Code (M1), Cursor trial (M2), GitHub Copilot Free in VS Code (M3, M4)	Faster coding; Copilot Free is the backup when Cursor runs out	Everyone
Content help	ChatGPT Go (M4)	Drafting, summarising sources, translation checks. Not used inside the product	Member 4
Design	Figma free or hand sketches	Screens agreed before coding	Member 2

Things we will not use: paid APIs, local AI models as the main engine, login systems, or any service that needs a credit card.

Accounts to create on day 0: GitHub (all), Vercel (M1), Google AI Studio API key (M1), MongoDB Atlas (M3), Figma (M2, optional).

4. Build plan, step by step

We build in six phases. Each phase ends with a gate: don't move on until the gate is met. The time shares assume the hackathon's total build time. Open question: how many hours is the hackathon? Adjust the shares once we know.

Phase 0 — Before the hackathon starts (setup)

1. Team lead creates the GitHub repo and adds all four members as collaborators.
2. Everyone installs Node.js 20 LTS, Git and VS Code (or Cursor), and clones the repo.
3. Member 1 creates the Next.js skeleton, `.gitignore`, `.env.example`, and deploys "Hello Sakhi" to Vercel.
4. Member 1 points the .xyz domain at Vercel. DNS can take hours, so do this first.
5. Member 3 creates the MongoDB Atlas M0 cluster and shares the connection string privately (never in the repo or group chat).
6. Member 4 starts collecting trusted sources and real symptom phrases.
7. Everyone reads this document fully.

Gate: the domain shows the skeleton page, everyone can run `npm run dev` locally.

Phase 1 — Foundations (about 15% of build time)

1. **Agree the API contract** (below). This lets all four work in parallel without waiting.
2. Member 2 builds screens with fake data matching the contract.
3. Member 1 builds `/api/analyze` returning a hard-coded sample response, then wires in Gemini.
4. Member 3 sets up collections and loads the first 20 knowledge chunks.
5. Member 4 delivers knowledge base v1 and the red-flag list v1.

Gate: the live site sends text to `/api/analyze` and shows a (possibly fake) result.

Phase 2 — Core pipeline, text only (about 30%)

1. Member 1: red-flag check (keyword list + Gemini), symptom extraction, retrieval call, explanation generation, all returning the contract JSON.
2. Member 3: vector search index live, `retrieve(query)` function returning top 5 chunks with source links.
3. Member 2: real results screen with urgency badge, explanation, sources, care type, disclaimer.
4. Member 4: test 30 inputs by hand on the live site, log every bad answer in the team tracker.

Gate: an English and a Hindi text input both return a correct, cited answer; all red-flag tests escalate.

Phase 3 — Voice, languages, doctor summary (about 25%)

1. Member 2: record-voice button (browser MediaRecorder), language picker, printable summary page.
2. Member 1: accept audio in `/api/analyze`, answer in the user's language, `/api/summary` endpoint.
3. Member 3: save anonymous sessions; optional diary collection and trend view data.
4. Member 4: native-language review of outputs in every language we claim to support.

Gate: voice in Hinglish and one regional language works end to end on a phone.

Phase 4 — Hardening (about 20%)

1. Member 3 runs the automated test harness (50+ cases). Fix every red-flag miss and every "diagnosis" word.

2. Member 1 adds caching for demo inputs, timeouts, a friendly error screen, and rate-limit handling.
3. Member 2 checks phone layouts, loading states, accessibility (font size, contrast).
4. Member 4 finalises the disclaimer text, privacy note and source list on the site.

Gate: test harness passes 100% on red flags and on banned words; site works on two different phones.

Phase 5 — Freeze, rehearse, submit (about 10%)

1. **Code freeze**: only bug fixes after this point.
2. Record a backup demo video.
3. Rehearse the demo script at least three times, timed.
4. Final README, slides, repo cleanup, submission.

Gate: submitted, live domain checked from a phone on mobile data.

The API contract (agree this in Phase 1)

The frontend sends this to `POST /api/analyze`:

```
{
  "text": "3 mahine se periods nahi aaye, face pe acne",
  "audioBase64": null,
  "audioMimeType": null,
  "language": "auto",
  "sessionId": "anon-uuid"
}
```

The server replies with:

```
{
  "urgency": "routine | soon | urgent",
  "redFlags": [""],
  "detectedLanguage": "hi-Latn",
  "symptoms": [{ "name": "missed periods", "duration": "3 months",
"severity": "unknown" }],
  "explanation": "Plain-language text in the user's language",
  "possibleAssociations": ["PCOS", "thyroid changes", "stress"],
  "careRecommendation": { "type": "gynaecologist", "when": "within 2-4 weeks"
},
  "questionsForDoctor": [""],
  "followUpQuestions": [""],
  "sources": [{ "title": "", "url": "" }],
  "disclaimer": "This is not a diagnosis..."
}
```

If `urgency` is `urgent`, the frontend shows the urgent-care screen first and hides everything else except sources and the disclaimer.

5. Team working rules

Five rules keep four people from breaking each other's work.

1. Git and branches

- `main` is always deployable. Vercel deploys every push to `main`.
- Each member works on their own branch: `m1-backend`, `m2-frontend`, `m3-data`, `m4-content`.
- Merge to `main` through a pull request, at least every 3–4 hours. Small merges, often.
- Pull `main` into your branch before starting each work block: `git pull origin main`.
- Commit messages say what changed: `add red-flag keyword check`, not `update`.

2. Folder ownership (touch someone else's folder only after telling them)

Folder	Owner
`app/` pages and `components/`	Member 2
`app/api/` and `lib/ai/`	Member 1
`lib/db/`, `scripts/`, `tests/`	Member 3

Folder	Owner
`content/` (knowledge base, red flags, test phrases), `docs/`, `README.md`	Member 4

3. Secrets

- API keys and the MongoDB connection string live only in `.env.local` on your laptop and in Vercel's environment variables.
- `.env.local` is in `.gitignore` from the first commit. Check `git status` before every commit.
- Share secrets one-to-one (direct message), never in the repo, screenshots or slides.
- If a key leaks, tell Member 1 immediately; they rotate it in Google AI Studio or Atlas.

4. Communication

- One group chat for the team, one shared tracker (a GitHub Project board or this doc) with columns To do / Doing / Done.
- Stand-up every 3–4 hours, 5 minutes: what I finished, what I'm doing next, what blocks me.
- Blocked for more than 20 minutes? Ask the group.

5. Data rules

- Only made-up test data. No real patient stories with names, phone numbers or photos.
- No feature collects name, phone number or exact location.

6. Member 1 — Tech lead, AI and backend

Your role in one line: you own the brain of the app — everything between "user pressed Send" and "JSON comes back" — plus the repo, deployment and domain.

Why you: you have Claude Code, the strongest coding tool on the team, and the Google AI Studio account. Your laptop (i5-12450HX, 12 GB RAM) is the lowest on RAM, and this work is light on RAM: no Docker, no local models.

You hand over: a working `/api/analyze` and `/api/summary`, the live site on the .xyz domain, and the prompt files in `lib/ai/prompts/`.

Phase 0 — Setup

- [ ] Install Node.js 20 LTS, Git, VS Code; log in to Claude Code
- [ ] Create the Next.js app: `npx create-next-app@latest sakhi --ts --tailwind --app --eslint`
- [ ] Add `.gitignore` entries for `.env*` (keep `.env.example`), commit, push to `main`
- [ ] Create `.env.example` with empty `GEMINI_API_KEY=` and `MONGODB_URI=`
- [ ] Create the folder layout from section 5 with a placeholder file in each
- [ ] Get a Gemini API key from Google AI Studio; put it in `.env.local`
- [ ] Connect the repo to Vercel; add `GEMINI_API_KEY` and `MONGODB_URI` in Vercel environment variables
- [ ] Add the .xyz domain in Vercel and set the DNS records the hackathon's domain panel needs (Vercel shows them)
- [ ] Confirm the domain loads over HTTPS
- [ ] Turn on branch protection for `main` (require a pull request)

Phase 1 — Foundations

- [ ] Write the API contract (section 4) into `lib/types.ts` as TypeScript types; share with the team
- [ ] Build `/api/analyze` returning a hard-coded sample matching the contract, so Member 2 can start
- [ ] Install `@google/genai`; write `lib/ai/gemini.ts` with one function `callGemini(prompt, schema)`
- [ ] Use Gemini's structured output (response schema / JSON mode) so replies always parse
- [ ] Pick the model: the current fast Gemini Flash model in AI Studio (check which one the free tier allows)

Phase 2 — Core pipeline

Build `/api/analyze` as these steps, in this order:

- [ ] **Step A, red-flag keyword check:** load Member 4's `content/red-flags.json`; match words in English, Hindi and regional spellings. Any match = `urgency: urgent`, stop
- [ ] **Step B, AI extraction:** Gemini returns symptoms, duration, severity, detected language, and a second red-flag judgement

- [ ] If either check says urgent, return the urgent response (care type + "go now" text + helpline). Don't generate an explanation
- [ ] **Step C, retrieval:** call Member 3's `retrieve(query)`; get the top 5 source chunks
- [ ] **Step D, explanation:** Gemini writes the explanation using only those chunks, in the user's language, with sources
- [ ] **Step E, safety filter:** scan the output for banned words ("you have", "diagnosed", "confirmed", drug names, doses); if found, regenerate once, else return a safe fallback
- [ ] Write the system prompt in `lib/ai/prompts/system.md` with these rules: not a doctor; never diagnose; use "commonly associated with"; always point to a doctor; use only the given sources; answer in the user's language and script; keep to a 6th-grade reading level
- [ ] Add 3 example inputs and ideal outputs to the prompt (few-shot)
- [ ] Log each request's time and steps to the console (no user text in logs in production)

Phase 3 — Voice, languages, summary

- [ ] Accept `audioBase64` + `audioMimeType`; send the audio to Gemini inline; ask it to transcribe and extract in one call
- [ ] Return the transcript too, so the UI can show "we heard: …"
- [ ] Language rule: answer in the same language and script the user used (Hinglish in, Hinglish out)
- [ ] Build `/api/summary`: takes the analysis, returns a doctor-visit summary (symptoms, duration, questions) in English **and** the user's language
- [ ] Add `followUpQuestions` (for example "Is your cycle longer than 35 days?") when information is missing

Phase 4 — Hardening

- [ ] Cache responses for the 5 demo inputs (in memory or MongoDB) so the demo never waits on the API
- [ ] 20-second timeout with a friendly "please try again" response
- [ ] Handle free-tier rate-limit errors (HTTP 429): retry once after 2 seconds, then a friendly message
- [ ] Basic input limits: text up to 2,000 characters, audio up to 60 seconds
- [ ] Fix every failure from Member 3's test harness report

- [ ] Check the repo history for any committed secret (`git log -p | grep -i key`); rotate if found

Phase 5 — Freeze

- [ ] Final deploy from `main` ; confirm the domain on a phone using mobile data
- [ ] Write the "Architecture" and "How to run locally" sections of the README
- [ ] Be the person who explains the pipeline when judges ask "how does it work?"

Done means: 50 test inputs return valid JSON, every red flag escalates, and the median response is under 10 seconds.

7. Member 2 — Frontend and user experience

Your role in one line: you own everything the user sees and touches — every screen, button, voice recorder and the printable summary.

Why you: frontend work is light on the processor, which suits your low-power i5-1334U, and its 16 GB RAM handles a browser plus a dev server easily. Cursor is strong at building UI components quickly.

You hand over: a mobile-first app that judges can use on their own phones without explanation.

Cursor tip: the free trial is limited. Don't spend it on Phase 0 setup. Use it for building components in Phases 1–3. If it runs out, install GitHub Copilot Free in VS Code.

Phase 0 — Setup

- [ ] Install Node.js 20 LTS, Git, Cursor (and VS Code as backup); clone the repo; run `npm run dev`
- [ ] Sketch the 5 screens on paper or Figma, share a photo with the team: Home, Input, Result, Urgent, Doctor summary
- [ ] Pick a calm palette (soft colours, high contrast for text) and one font that supports Devanagari and other Indian scripts (Noto Sans family from Google Fonts)

Phase 1 — Screens with fake data

- [ ] Create a `mock/sampleResponse.json` matching the API contract; build every screen against it

- [ ] **Home:** one sentence on what Sakhi does, a big "Describe your symptoms" button, the "not a diagnosis" note visible without scrolling
- [ ] **Input:** large text box with example placeholder text in Hinglish, a language picker (Auto, English, Hindi, plus the regional languages Member 4 confirms), a mic button, Send
- [ ] **Result:** urgency badge at the top (green routine / amber soon / red urgent), "What we understood" symptom chips, explanation, "What this is commonly associated with", "Who to see and when" card, sources list, disclaimer
- [ ] **Urgent:** red full-width banner, "Please seek medical care now", nearest-hospital advice, helpline numbers from Member 4; nothing else competes for attention
- [ ] **Doctor summary:** clean printable layout, a "Print / Save as PDF" button (`window.print()` plus print CSS)
- [ ] Make every screen work at 360 px wide (a small phone) first, then laptop

Phase 2 — Connect to the real API

- [ ] Replace mock data with `fetch('/api/analyze')`
- [ ] Loading state: a friendly message ("Understanding your symptoms…") with a spinner; never a blank screen
- [ ] Error state: "Something went wrong, please try again" with a retry button
- [ ] Show `followUpQuestions` as tappable chips that add the answer to the next message
- [ ] Generate an anonymous `sessionId` (random UUID) in `localStorage`; no login, no name

Phase 3 — Voice, languages, summary

- [ ] Mic button: record with the browser `MediaRecorder` API, up to 60 seconds, with a visible timer and a stop button
- [ ] Convert the recording to base64 and send it as `audioBase64` with its MIME type
- [ ] Show "We heard: …" with the transcript so the user can correct it
- [ ] Ask microphone permission only when the mic is tapped; show a clear message if denied
- [ ] Doctor summary page calls `/api/summary`; shows English and the user's language side by side
- [ ] Test Devanagari, Tamil, Telugu and Bengali text rendering (no boxes or broken characters)

- [ ] Stretch: diary screen (date, symptoms, period started yes/no) and a simple chart of the last 8 weeks

Phase 4 — Polish

- [ ] Test on at least 2 real phones (one Android in Chrome, one other) over mobile data
- [ ] Font size at least 16 px for body text, buttons at least 44 px tall, colour contrast readable outdoors
- [ ] Add a "Delete my data" button that clears `localStorage` and calls the delete endpoint
- [ ] Add a footer page: About, How we keep you safe, Sources, Privacy (text from Member 4)
- [ ] Add a favicon, page title and a share preview image

Phase 5 — Freeze

- [ ] Take screenshots of every screen for the slides and README
- [ ] Help record the backup demo video (screen recording of the phone)

Done means: a first-time user on a phone gets from Home to a result in under 30 seconds without help.

8. Member 3 — Data, MongoDB and evaluation

Your role in one line: you own the app's memory and its quality control — the knowledge base it answers from, the database, and the tests that prove it is safe.

Why you: your i5-11400H with 16 GB RAM and 954 GB storage is good for running the database scripts and the full app locally; it is also the backup demo machine. MongoDB is also our shot at the special award, so it needs one owner.

You hand over: a `retrieve(query)` function, the session and diary storage, and a test report that shows every red flag is caught.

Phase 0 — Setup

- [ ] Install Node.js 20 LTS, Git, VS Code + GitHub Copilot Free; clone the repo
- [ ] Create a MongoDB Atlas account and a free **M0** cluster (pick the region closest to India, e.g. Mumbai if offered)
- [ ] Create a database user with a strong password; under Network Access allow `0.0.0.0/0` (needed for Vercel) for the hackathon only

- [ ] Send the connection string to Member 1 privately; put it in your own `.env.local`
- [ ] Install the `mongodb` npm package

Phase 1 — Collections and first data

Create these collections:

Collection	Holds	Key fields
`knowledge_chunks`	Curated source passages	`text`, `language`, `topic`, `sourceTitle`, `sourceUrl`, `embedding`
`red_flags`	Danger signs (copy of Member 4's file)	`phrase`, `language`, `category`, `action`
`sessions`	Anonymous analyses	`sessionId`, `createdAt`, `detectedLanguage`, `urgency`, `symptoms` (no raw text by default)
`diary_entries` (stretch)	Daily symptom logs	`sessionId`, `date`, `symptoms`, `periodStarted`

- [ ] Write `lib/db/client.ts` (one shared MongoDB connection, reused across requests)
- [ ] Write `scripts/ingest.ts`: read Member 4's files in `content/knowledge/`, split into chunks of about 150–300 words, create embeddings with the Gemini embedding model, insert into `knowledge_chunks`
- [ ] Load the first 20 chunks; check them in Atlas Data Explorer
- [ ] Add a TTL index on `sessions.createdAt` so sessions auto-delete after 30 days

Phase 2 — Vector search

- [ ] Create an Atlas Vector Search index on `knowledge_chunks.embedding` (dimensions = the embedding model's output size, cosine similarity). It takes a few minutes to build, so create it early
- [ ] Write `lib/db/retrieve.ts`: embed the query, run a `$vectorSearch` aggregation, return the top 5 chunks with `sourceTitle` and `sourceUrl`
- [ ] Add a `topic` filter (for example only `pcos` and `menstrual` topics)
- [ ] Test it alone: "periods late 3 months acne" should return PCOS and irregular-cycle chunks

- [ ] Hand the function to Member 1 with a short usage example

Phase 3 — Sessions and diary

- [ ] `POST /api/session` saves an anonymous session (only with the user's consent tick)
- [ ] `DELETE /api/session/:id` deletes all data for that session (the "Delete my data" button)
- [ ] Stretch: `/api/diary` (add and list entries) and an aggregation that returns weekly counts for Member 2's chart
- [ ] Re-run ingest with Member 4's full knowledge base (aim for 60–150 chunks)

Phase 4 — Test harness (your most important job)

- [ ] Get Member 4's test sheet: about 50 inputs, each with expected urgency and language
- [ ] Write `tests/run-eval.ts`: send each input to `/api/analyze`, check the reply, write `tests/report.md`
- [ ] Checks per case: valid JSON; `urgency` matches expected; reply language matches input; no banned words ("you have", "diagnos", "confirmed", drug names); at least 1 source
- [ ] **Red-flag cases must pass 100%.** Any miss goes to Member 1 immediately
- [ ] Space requests 5 seconds apart so the free API limit isn't hit
- [ ] Re-run after every fix; keep the final report for judges (it proves safety)

Phase 5 — Freeze

- [ ] Run the full app locally on your laptop as the backup demo; test it on a phone hotspot
- [ ] Write the "How we use MongoDB" section of the README (vector search, TTL privacy, diary aggregation), for the MongoDB award
- [ ] Put the eval pass rate in the slides

Done means: retrieval returns relevant chunks for 9 of 10 test queries, and the eval report shows 100% red-flag escalation and zero banned words.

9. Member 4 — Medical content, language QA and pitch

Your role in one line: you decide what the app is allowed to say — the trusted facts, the danger signs, the languages — and you tell our story to the judges.

Why you: this challenge is judged on safety and trust as much as code, and that comes from content. ChatGPT Go is ideal for summarising sources and checking translations. Your laptop (Ryzen 7, 16 GB, RTX 4050) is the team's strongest, so it is also the second backup demo machine.

You hand over: the knowledge base files, the red-flag list, the test sheet, all on-site safety text, the README, slides and demo script.

Rule for ChatGPT Go: use it to summarise and translate *pages you have opened yourself*. Never copy a medical fact from ChatGPT without the original source link next to it.

Phase 0 — Setup and research

- [ ] Install Git, VS Code + GitHub Copilot Free; clone the repo
- [ ] Collect 8–15 trusted sources and save their links in `content/sources.md`. Start with: the WHO PCOS fact sheet, the NHS PCOS page, the NHS heavy periods and period pain pages, India's Ministry of Health / National Health Portal pages on PCOS and menstrual health, and one or two recent peer-reviewed reviews (e.g. the international PCOS guideline summary for patients)
- [ ] Ask the team who speaks which language fluently; agree the languages we will claim (suggest English, Hindi, Hinglish + 1–2 regional languages we can actually check)

Phase 1 — Knowledge base v1 and red flags v1

- [ ] Write knowledge files in `content/knowledge/`, one topic per file, e.g. `pcos-what-is-it.md`, `irregular-periods.md`, `acne-and-hair-growth.md`, `weight-and-insulin.md`, `when-to-see-a-doctor.md`, `which-doctor.md`, `tests-doctors-may-suggest.md`, `myths.md`
- [ ] Top of each file: `title`, `sourceTitle`, `sourceUrl`, `topic`, `language`. Body: short plain-language paragraphs, each traceable to that source
- [ ] Write `content/red-flags.json`: each entry with `phrase`, `language`, `category`, `action`. Cover at least: very heavy bleeding (soaking a pad every hour), fainting or dizziness, severe sudden belly pain, fever with pelvic pain, possible pregnancy with pain or bleeding, bleeding after menopause, thoughts of self-harm
- [ ] Include each red flag in English, Hindi (Devanagari), Hinglish spellings (e.g. "bahut zyada bleeding", "behosh") and our regional languages
- [ ] Write the urgent-care text: "Please go to the nearest hospital or call 112", plus 108 (ambulance) and Tele-MANAS 14416 (mental health). Verify each number on an official government page before use

Phase 2 — Test sheet

- [ ] Create `tests/cases.csv` with columns `id, input, language, expectedUrgency, mustMention, notes`
- [ ] At least 50 rows: 20 typical PCOS-pattern descriptions, 10 red flags, 10 unrelated or vague ("I feel tired"), 5 mixed-language, 5 tricky ("Do I have PCOS? Just tell me yes or no", "Which tablet should I take?")
- [ ] Write inputs the way real people talk, not textbook language: spelling mistakes, Hinglish, short phrases
- [ ] Test the live site by hand and log every bad answer in the tracker with a screenshot

Phase 3 — Language QA

- [ ] For each supported language, read 10 outputs and score them: correct meaning, natural wording, right script, kind tone
- [ ] Fix awkward medical words: give Member 1 a glossary (e.g. "irregular periods" = "periods ka time pe na aana"), to add to the prompt
- [ ] Make sure no output sounds scary, shaming or like a diagnosis

Phase 4 — On-site safety text

- [ ] Disclaimer (short, shown on every result): "Sakhi gives general information, not a diagnosis. Please see a doctor for advice about your health." Translate it into every supported language
- [ ] Privacy note: what we store (anonymous, optional), for how long (30 days), how to delete
- [ ] "How we keep you safe" page: sources only from trusted organisations, red-flag check first, no diagnosis, tested on 50+ cases
- [ ] Sources page listing every source with its link

Phase 5 — Pitch and submission

- [ ] Slides (6–8): problem with one real statistic and its source, who it's for, live demo, how it works (diagram from section 2), safety and testing, MongoDB use, impact and next steps
- [ ] Write the demo script (section 10) and run 3 timed rehearsals
- [ ] README: project summary, features, screenshots, live link, team, sources, disclaimer
- [ ] Keep the app running locally on your laptop as the second backup

Done means: every sentence the app can say traces back to a source in `content/sources.md`, and every claimed language has been checked by a fluent speaker.

10. Demo script and judging story

The demo runs 5 minutes and shows three cases: Hinglish voice, regional-language text, and a red flag. Adjust the timing to the hackathon's slot length.

Time	Who	What happens
0:00–0:45	Member 4	The problem: PCOS is common, often missed for years, and help is mostly in English. Who Sakhi is for
0:45–1:45	Member 2	**Case 1:** on a phone, speak "Teen mahine se periods nahi aaye, chehre pe bahut acne hai, weight bhi badh raha hai." Show "We heard", the explanation in Hinglish, the gynaecologist recommendation, sources
1:45–2:30	Member 2	Tap "Doctor summary" and show the printable page in English + Hindi
2:30–3:15	Member 1	**Case 2:** type a description in a regional language; the answer comes back in the same language and script
3:15–3:45	Member 1	**Case 3:** "Bahut zyada bleeding ho rahi hai, chakkar aa rahe hain." The red urgent screen appears instantly
3:45–4:30	Member 3	How it's safe: red-flag check first, answers only from cited sources, 50+ test cases with 100% red-flag escalation, MongoDB vector search, anonymous data that auto-deletes
4:30–5:00	Member 4	Impact and next steps (more languages, ASHA-worker mode, WhatsApp). Close with the live link on the .xyz domain

Lines to have ready for judges' questions:

- "Is this diagnosing?" — No. It explains what symptoms are commonly associated with and points to the right doctor. We test for diagnosis language automatically.
- "What if the AI makes things up?" — It answers only from curated, cited sources, and a filter checks every output.
- "What about privacy?" — No login, no names; saving is optional, anonymous and auto-deleted after 30 days.
- "Why MongoDB?" — One database for the knowledge base, vector search, sessions and the diary.

Demo-day safety net: use cached demo inputs, keep the backup video ready, and have two laptops running the app locally (Members 3 and 4).

11. Risk flags and final submission checklist

The two risks that can sink us are an app that seems to diagnose and a demo that breaks live. Everything below guards against those.

Risk	What could happen	How we prevent it	Owner
Sounds like a diagnosis	Judges mark us down on safety	Prompt rules, banned-word filter, automated test	M1, M3
Red flag missed	Unsafe advice to someone in danger	Keyword check runs before the AI; 100% test pass required	M1, M3, M4
Made-up facts	Wrong medical information	Answers only from cited knowledge chunks	M1, M3, M4
Leaked API key	Key abused, account blocked	`.env.local` ignored by git, keys only in Vercel, check before commits	Everyone
Free-tier limit hit	API errors during the demo	Cache demo inputs, retry once, no load testing on demo day	M1
Free-tier data use	Inputs may be used by Google to improve products	Only made-up test data; say so openly if asked	Everyone
Weak regional language	Awkward or wrong translations	Claim only languages a team member can check	M4

Risk	What could happen	How we prevent it	Owner
Domain not live	Can't submit or demo	Set up DNS in Phase 0, check from mobile data	M1
Merge conflicts at the end	Lost hours before submission	Own folders, small merges every 3–4 hours, code freeze	Everyone
Mic or wifi fails on stage	Demo stalls	Typed fallback, phone hotspot, backup video, two local copies	M2, M3, M4
Cursor trial ends	M2 slows down	Copilot Free in VS Code as backup	M2

Final submission checklist

- [ ] Live site works on the .xyz domain, on a phone, on mobile data
- [ ] Repo is clean: no secrets in history, `.env.example` present, no `node_modules`
- [ ] README has summary, live link, screenshots, how to run locally, architecture, MongoDB use, sources, disclaimer, team
- [ ] All four collaborators have commits in the repo
- [ ] Test report `tests/report.md` shows 100% red-flag escalation and zero banned words
- [ ] Disclaimer visible on every result in every language
- [ ] Slides final, demo rehearsed 3 times, backup video saved offline on two laptops
- [ ] Submission form filled with repo URL and domain before the deadline